

SECTION 35 — RAPID-FIRE INTERVIEW QUESTIONS

Purpose: Dense, expert-level Q&A pairs designed for rapid review before a Senior SDET & QA Architect interview (Deloitte, FAANG, Big 4). Each answer is technically precise, covering traps and gotchas that trip up senior engineers. Read aloud, quiz yourself, and time your answers to under 30 seconds per pair.

35.1 Java Rapid Fire

Q: What is the difference between `==` and `.equals()` for String comparison in Java?

A: `==` compares object references (memory addresses), while `.equals()` compares character content. String literals are interned in the String Pool, so `"abc" == "abc"` can be `true`, but `new String("abc") == new String("abc")` is always `false`. Always use `.equals()` for value comparison.

Q: What is the difference between `final`, `finally`, and `finalize()`?

A: `final` is a modifier that makes variables constant, methods non-overridable, and classes non-inheritable. `finally` is a block in try-catch that always executes regardless of exception outcome — critical for resource cleanup. `finalize()` is a deprecated `Object` method called by the GC before object destruction; avoid relying on it due to non-deterministic GC scheduling.

Q: Explain method overloading vs. method overriding. Which is resolved at compile time vs. runtime?

A: Overloading is compile-time polymorphism — same method name, different parameter signatures in the same class. Overriding is runtime polymorphism — subclass provides a specific implementation of a superclass method. The JVM uses the actual runtime type of the object (via vtable dispatch) to resolve overriding, while overloading is resolved by the compiler based on static type.

Q: What is autoboxing and where can it cause a `NullPointerException`?

A: Autoboxing is the automatic conversion between primitives (`int`) and their wrapper classes (`Integer`). An NPE occurs when unboxing a `null` wrapper: `Integer x = null; int y = x;` throws NPE at the unboxing step. This is especially dangerous in collections: `Map<String, Integer> map = new HashMap<>(); int val = map.get("missing");` — `get()` returns `null`, unboxing explodes.

Q: What is the difference between `ArrayList` and `LinkedList`? When do you choose each?

A: `ArrayList` is backed by a dynamic array — $O(1)$ random access, $O(n)$ insertion/deletion in the middle. `LinkedList` is a doubly-linked list — $O(n)$ access, $O(1)$ insertion/deletion at known positions. Choose `ArrayList` for read-heavy workloads and `LinkedList` for queue/dequeue operations. In practice, `ArrayList` wins most benchmarks due to cache locality.

Q: What is a `volatile` keyword and when is it insufficient for thread safety?

A: `volatile` guarantees visibility — writes to a volatile variable are immediately flushed to main memory and reads always come from main memory, preventing CPU cache staleness. It does NOT guarantee atomicity. `volatile int`

`count; count++;` is not thread-safe because increment is a read-modify-write compound operation. Use `AtomicInteger` or `synchronized` for compound atomicity.

Q: What does `synchronized` actually do at the JVM level?

A: `synchronized` acquires a monitor lock (intrinsic lock) on the specified object. Only one thread can hold the monitor at a time; others block on the lock's wait-set. It establishes a happens-before relationship — all writes made before releasing the lock are visible to the thread that subsequently acquires it. Synchronized methods use `this` as the monitor; static synchronized methods use the `Class` object.

Q: Explain the difference between `Callable` and `Runnable`.

A: `Runnable` has a `run()` method that returns `void` and cannot throw checked exceptions. `Callable<V>` has a `call()` method that returns a value of type `V` and can throw checked exceptions. `Callable` is used with `ExecutorService.submit()`, which returns a `Future<V>` to retrieve the result. Use `Callable` whenever your concurrent task needs to return data or propagate checked exceptions.

Q: What is the difference between `throw` and `throws`?

A: `throw` is the statement used to actually raise an exception instance: `throw new IllegalArgumentException("bad input")`. `throws` is a method signature declaration indicating the method may propagate checked exceptions: `void parse() throws IOException`. Forgetting to declare a checked exception with `throws` causes a compile error; unchecked exceptions (`RuntimeException` subclasses) don't require `throws`.

Q: What is a functional interface and give three examples from the JDK?

A: A functional interface has exactly one abstract method and is annotated with `@FunctionalInterface` (optional but recommended). Examples: `Runnable` (no args, no return), `Callable<V>` (no args, returns `V`), `Predicate<T>` (takes `T`, returns boolean), `Function<T,R>` (takes `T`, returns `R`), `Consumer<T>` (takes `T`, returns void). They enable lambda expressions and method references.

Q: What is the difference between `Optional.orElse()` and `Optional.orElseGet()`?

A: `orElse(value)` always evaluates its argument, even if the `Optional` is present — this is wasteful if the fallback is expensive. `orElseGet(supplier)` is lazy — the `Supplier` is only called when the `Optional` is empty. Always prefer `orElseGet(() -> expensiveComputation())` over `orElse(expensiveComputation())` for performance-sensitive code.

Q: What is the memory model difference between stack and heap in Java?

A: The stack holds method frames (local variables, references, return addresses) and is thread-private — each thread has its own stack. The heap is shared across all threads and holds object instances and arrays. Stack allocation is fast (pointer bump), heap allocation involves GC management. Primitives live on the stack; objects always live on the heap even if referenced by a local variable.

Q: What is the difference between checked and unchecked exceptions? When should you use each?

A: Checked exceptions (`extends Exception`) must be declared in `throws` or caught — they model recoverable conditions the caller should handle (e.g., `IOException`, `SQLException`). Unchecked exceptions (`extends RuntimeException`) represent programming errors (e.g., `NullPointerException`, `IllegalArgumentException`).

Use checked for external resource failures; use unchecked for API contract violations where the caller cannot reasonably recover.

Q: Explain `try-with-resources`. What interface must the resource implement?

A: Try-with-resources (Java 7+) automatically calls `close()` on declared resources at the end of the try block, even if an exception is thrown. The resource must implement `AutoCloseable` (or its subinterface `Closeable`). If both the try body and `close()` throw, the close exception is suppressed and can be retrieved via `Throwable.getSuppressed()`.

Q: What is the difference between `String`, `StringBuilder`, and `StringBuffer`?

A: `String` is immutable — every concatenation creates a new object, causing GC pressure in loops. `StringBuilder` is mutable and NOT thread-safe — use in single-threaded contexts. `StringBuffer` is mutable and thread-safe via `synchronized` methods — but significantly slower. In test automation, always use `StringBuilder` for building dynamic XPath, SQL queries, or report strings in loops.

Q: What does `static` mean on an inner class?

A: A `static` nested class does not hold a reference to the enclosing instance — it's essentially a top-level class namespaced inside another. A non-static inner class holds an implicit reference to its enclosing instance, which can cause memory leaks if the inner class outlives the outer. In test framework design, use static nested classes for helper builder patterns (e.g., `RequestBuilder.Builder`).

Q: What is the difference between `HashMap` and `ConcurrentHashMap`?

A: `HashMap` is not thread-safe — concurrent modifications can cause infinite loops (Java 7 rehashing bug) or data corruption. `ConcurrentHashMap` uses segment-level locking (Java 7) or CAS + bin-level synchronization (Java 8+) for thread-safe, high-concurrency access without locking the entire map. `ConcurrentHashMap` does not allow `null` keys or values; `HashMap` does.

Q: What is a deadlock and how do you prevent it?

A: A deadlock occurs when two or more threads each hold a lock and wait for the other's lock, creating a circular dependency. Prevention strategies: (1) Lock ordering — always acquire locks in the same global order. (2) Lock timeout — use `tryLock(timeout)` from `ReentrantLock`. (3) Avoid nested locks. (4) Use higher-level concurrency primitives like `java.util.concurrent` classes that eliminate manual locking.

Q: What is the difference between `Iterator` and `ListIterator`?

A: `Iterator` supports forward-only traversal (`hasNext()`, `next()`, `remove()`). `ListIterator` extends `Iterator` with bidirectional traversal (`hasPrevious()`, `previous()`), index access (`nextIndex()`, `previousIndex()`), and mutation (`add()`, `set()`). `ListIterator` is only available for `List` implementations.

Q: What is `instanceof` pattern matching (Java 16+) and why is it useful in test code?

A: Pattern matching for `instanceof` combines the type check and cast: `if (obj instanceof String s) { s.toUpperCase(); }` — no separate cast needed. In test code this eliminates boilerplate when processing heterogeneous response objects, assertion results, or polymorphic page object hierarchies, reducing `ClassCastException` risk.

35.2 Collections Rapid Fire

Q: What is the contract between `hashCode()` and `equals()` in Java?

A: If `a.equals(b)` is `true`, then `a.hashCode()` MUST equal `b.hashCode()`. The converse is not required — hash collisions are allowed. Violating this contract breaks `HashMap`, `HashSet`, and `Hashtable` — objects that are logically equal will not be found in hash-based collections. Always override both or neither.

Q: What is the load factor in `HashMap` and what happens when it is exceeded?

A: The default load factor is 0.75 — when the number of entries exceeds `capacity × loadFactor`, the map is resized (rehashed) to double capacity. Rehashing is $O(n)$ and rebuilds the internal array and re-places all entries. A lower load factor reduces collisions but wastes memory; higher load factor saves memory but increases collision chains and degrades lookup from $O(1)$ toward $O(n)$.

Q: What is the difference between `fail-fast` and `fail-safe` iterators?

A: Fail-fast iterators (e.g., `ArrayList`, `HashMap`) throw `ConcurrentModificationException` if the collection is structurally modified during iteration (detected via `modCount`). Fail-safe iterators (e.g., `CopyOnWriteArrayList`, `ConcurrentHashMap`) iterate over a snapshot or use internal mechanisms that tolerate concurrent modification, but may not reflect the latest state.

Q: When would you use `TreeMap` over `HashMap`?

A: Use `TreeMap` when you need keys in natural sorted order or a custom `Comparator` order. `TreeMap` is backed by a Red-Black tree — $O(\log n)$ get/put/remove. `HashMap` is $O(1)$ average. Use `TreeMap` for range queries (`subMap()`, `headMap()`, `tailMap()`), finding floor/ceiling keys, or when iteration order must be deterministic and sorted (e.g., generating sorted test reports by test name).

Q: What is the difference between `Comparable` and `Comparator`?

A: `Comparable<T>` is implemented by the class itself (`compareTo()`) to define its natural ordering — intrusive design. `Comparator<T>` is an external strategy object that defines an ordering without modifying the class — non-intrusive. Use `Comparator` when you need multiple sort orders for the same class (e.g., sort test results by duration, by name, or by status) or when you cannot modify the class.

Q: What are the time complexities of `PriorityQueue` operations?

A: `add()` / `offer()` — $O(\log n)$ (sift-up in min-heap). `poll()` (remove minimum) — $O(\log n)$ (sift-down). `peek()` (view minimum) — $O(1)$. `remove(Object)` — $O(n)$ (linear scan then $O(\log n)$ sift). `PriorityQueue` does NOT guarantee sorted iteration — only the head is guaranteed to be the minimum. Use for implementing test scheduling by priority or shortest-path in automation graph analysis.

Q: What is `LinkedHashMap` and what ordering does it provide?

A: `LinkedHashMap` extends `HashMap` with a doubly-linked list connecting all entries in insertion order (default) or access order (LRU — set via constructor flag). Useful in test automation for maintaining ordered test parameter maps, building LRU caches for expensive resource setup, or ensuring deterministic iteration in assertion comparisons.

Q: Explain the internal structure of `ArrayList`. What happens during `add()` when capacity is full?

A: `ArrayList` is backed by an `Object[]` with a default initial capacity of 10. When full, `add()` triggers `grow()`,

which creates a new array at 1.5× the current capacity (`oldCapacity + (oldCapacity >> 1)`) and copies all elements via `Arrays.copyOf()` . The amortized cost of `add()` is $O(1)$. Pre-sizing with `new ArrayList<>(expectedSize)` eliminates resize overhead in performance-critical loops.

Q: What is `Collections.unmodifiableList()` vs. `List.of()` ?

A: `Collections.unmodifiableList()` wraps an existing list — the underlying list can still be modified by holding a reference to it, changes are visible through the wrapper. `List.of()` (Java 9+) creates a truly immutable list — no `null` elements, no structural modification, no set operations, and the list has value-based equality. Use `List.of()` for truly immutable test data sets.

Q: What is the difference between `Stack` and `Deque` for stack operations?

A: `Stack` extends `Vector` (synchronized, legacy, poor performance). The Java docs explicitly recommend using `Deque` implementations (`ArrayDeque`) instead. `ArrayDeque` as a stack (`push()` / `pop()` / `peek()`) has no synchronization overhead, no legacy baggage, and better cache performance. Use `Deque<T> stack = new ArrayDeque<>()` — never `Stack<T>` .

Q: How does `EnumSet` differ from `HashSet<MyEnum>` , and when is it preferred?

A: `EnumSet` is internally represented as a bit vector (one `long` per 64 enum constants) — all operations are $O(1)$ bitwise operations with near-zero memory overhead. `HashSet<MyEnum>` uses the standard hash table overhead (~48 bytes per entry). Use `EnumSet` whenever your set contains values from a single enum — it is dramatically faster and more memory-efficient for test status flags, feature toggles, or environment config sets.

Q: What is a `WeakHashMap` and what is a practical use case?

A: `WeakHashMap` holds weak references to keys — when a key has no strong references elsewhere, the GC can collect it and the entry is removed from the map. Practical use: caching test fixture objects keyed by test instances, where the cache should not prevent GC of completed test objects. Also used for canonical mapping and listener registries where entries should auto-expire.

Q: What is `CopyOnWriteArrayList` and what is its major performance caveat?

A: `CopyOnWriteArrayList` creates a fresh copy of the underlying array on every write operation (`add` , `set` , `remove`), making reads lock-free and safe for concurrent iteration. The major caveat: write operations are $O(n)$ due to array copying — it is only suitable for read-heavy, write-rare scenarios like test listener registries or event subscriber lists.

Q: What is the difference between `Collections.synchronizedList()` and `CopyOnWriteArrayList` ?

A: `synchronizedList()` wraps every method with `synchronized(mutex)` — reads and writes are serialized, and you must manually synchronize during iteration (`synchronized(list) { for(x : list) }`). `CopyOnWriteArrayList` allows lock-free reads and safe iteration without explicit synchronization but at the cost of $O(n)$ writes. For test result collection across threads, `CopyOnWriteArrayList` is usually simpler and safer.

Q: Explain the internal structure of `HashMap` in Java 8+ when a bucket exceeds 8 entries.

A: When a bucket's linked list exceeds 8 entries (and the table has ≥ 64 slots), the bucket is converted to a balanced Red-Black tree (a `TreeNode`), reducing worst-case lookup from $O(n)$ to $O(\log n)$. If the tree shrinks below 6 entries (during remove), it reverts to a linked list. This prevents denial-of-service attacks via hash collision flooding.

Q: What is `ArrayDeque` and why is it preferred over `LinkedList` for queue operations?

A: `ArrayDeque` is backed by a circular resizable array — it has better cache locality than `LinkedList` (no node pointer chasing), no per-node allocation overhead, and no nulls for head/tail pointers. For all queue/deque/stack operations, `ArrayDeque` outperforms `LinkedList` in benchmarks. `LinkedList` is preferred only when you need $O(1)$ mid-list insertion with a known iterator position.

35.3 Selenium Rapid Fire

Q: What is the difference between `findElement()` and `findElements()`? What do they return when nothing is found?

A: `findElement()` throws `NoSuchElementException` immediately if no element matches. `findElements()` returns an empty `List<WebElement>` — never throws — making it safer for conditional checks like `driver.findElements(By.id("modal")).isEmpty()`. Use `findElements().size() > 0` instead of try-catch on `findElement()` for existence checks.

Q: What is the difference between `driver.close()` and `driver.quit()`?

A: `close()` closes the currently focused browser window/tab but leaves the WebDriver session active. `quit()` terminates the entire WebDriver session, closes all windows, and kills the browser process. Always call `quit()` in `@AfterSuite` to prevent zombie browser processes in CI environments.

Q: Explain `StaleElementReferenceException` — when does it occur and how do you handle it?

A: It occurs when the DOM element a `WebElement` reference points to no longer exists in the DOM — typically after a page refresh, AJAX re-render, or navigation. Handle it by re-locating the element after the DOM mutation, or wrapping the interaction in a retry loop with `ExpectedConditions`. In fluent waits, `StaleElementReferenceException` can be whitelisted as an ignored exception.

Q: What is the difference between implicit wait and explicit wait? Can you use them together?

A: Implicit wait sets a global timeout for `findElement()` to poll for element presence. Explicit wait (`WebDriverWait`) waits for a specific condition (visibility, clickability) on a specific element. Using both together causes unpredictable wait durations — the effective wait can be the sum of both timeouts. Best practice: set implicit wait to 0 and use only explicit waits for deterministic behavior.

Q: What does `JavascriptExecutor` allow you to do that WebDriver cannot?

A: `JavascriptExecutor` executes arbitrary JavaScript in the browser context — useful for: scrolling elements into view, clicking elements that WebDriver refuses (intercepted clicks), reading/writing `localStorage` / `sessionStorage`, triggering custom DOM events, and bypassing front-end validation during test data setup. Use sparingly — JS clicks bypass browser event pipeline and may mask real user interaction bugs.

Q: What is `FluentWait` and how does it differ from `WebDriverWait`?

A: `WebDriverWait` is a concrete subclass of `FluentWait<WebDriver>` with `NoSuchElementException` ignored by default and a 500ms polling interval. `FluentWait` is the generic base — you configure polling interval, timeout, ignored exceptions, and a custom message yourself. `FluentWait` gives full control; use it when you need to ignore `StaleElementReferenceException` or custom polling intervals not available via `WebDriverWait`.

```
FluentWait<WebDriver> wait = new FluentWait<>(driver)
    .withTimeout(Duration.ofSeconds(30))
    .pollingEvery(Duration.ofMillis(200))
    .ignoring(StaleElementReferenceException.class)
    .withMessage("Element not stable after 30s");
```

Q: What is the `Page Object Model` and what problem does it solve architecturally?

A: POM encapsulates page structure (locators) and page behaviors (interactions) inside dedicated classes, decoupling test logic from UI implementation details. When the UI changes, only the Page Object needs updating, not every test. It promotes reusability, readability, and maintainability. The complementary `Page Factory` uses `@FindBy` annotations with lazy `WebElement` proxies initialized via `PageFactory.initElements()`.

Q: How do you handle a file upload in Selenium without using `Robot` or `AutoIT`?

A: Use `sendKeys()` on the `<input type="file">` element with the absolute file path — this directly sets the file input's value without opening the OS dialog: `fileInput.sendKeys("/path/to/file.pdf")`. This works even if the input is hidden; just send keys directly to it without clicking. `Robot` / `AutoIT` are OS-level and brittle; avoid them.

Q: What is `WebDriverManager` and what problem does it solve?

A: `WebDriverManager` (Bonigarcia) automates browser driver binary management — it detects the installed browser version, downloads the matching driver (chromedriver, geckodriver, etc.) from official repositories, caches it locally, and sets the system property. Eliminates the brittle manual process of matching chromedriver versions to Chrome versions in CI pipelines.

Q: How does Selenium Grid work architecturally?

A: Grid has a Hub (router) and Nodes (browsers). Tests send WebDriver commands to the Hub, which routes requests to Nodes matching the requested `DesiredCapabilities` / `Options`. Grid 4 uses a fully distributed architecture with components: Router, Distributor, Session Map, Event Bus, and Nodes communicating over a message bus. Supports parallel execution across multiple OS/browser combinations.

Q: What is `Actions` class used for and give three examples?

A: `Actions` builds a chain of complex user interactions — mouse and keyboard events. Examples: (1) `actions.moveToElement(el).click()` — hover then click. (2) `actions.dragAndDrop(source, target)` — drag-and-drop. (3) `actions.keyDown(Keys.CONTROL).sendKeys("a").keyUp(Keys.CONTROL)` — select all. Call `.perform()` to execute the chain. `Actions` uses `PointerInput` and `KeyInput` sequences internally.

Q: What is `SwitchTo` and when do you use it?

A: `driver.switchTo()` redirects WebDriver commands to a different context: `.frame(nameOrIndex)` for iframes, `.window(handle)` for tabs/popups, `.alert()` for browser dialogs, `.defaultContent()` to return to the main page. Forgetting to switch context before interacting with iframe content causes `NoSuchElementException` — the most common iframe-related bug.

Q: What causes `ElementNotInteractableException` and how do you resolve it?

A: The element is in the DOM and visible but cannot receive user input — it may be covered by another element, disabled, have `visibility:hidden`, or be off-screen. Resolution: (1) Scroll into view with `JavascriptExecutor`.

(2) Wait for the overlay to disappear. (3) Use `WebDriverWait` with `elementToBeClickable()`. (4) As a last resort, `JavaScriptExecutor.executeScript("arguments[0].click()", element)`.

Q: What is the difference between `By.xpath()` and `By.cssSelector()` for locating elements?

A: CSS selectors are generally faster (browser-native engine) and more readable. XPath is more powerful — supports text content matching (`contains(text(), 'Login')`), traversal from child to parent (ancestor axis), and complex predicates. CSS cannot traverse upward (no parent selector until CSS4) or select by text content. Prefer CSS for stability and speed; use XPath only when CSS is insufficient.

Q: How do you capture a screenshot in Selenium and attach it to a report?

A: `TakesScreenshot ts = (TakesScreenshot) driver; byte[] screenshot = ts.getScreenshotAs(OutputType.BYTES);` Attach the byte array to Allure: `Allure.addAttachment("Screenshot", "image/png", new ByteArrayInputStream(screenshot), ".png")` or use the `@Attachment` annotation in a utility method. Take screenshots in a TestNG `ITestListener.onTestFailure()` hook automatically.

Q: What is `DesiredCapabilities` vs. browser-specific `Options` classes?

A: `DesiredCapabilities` is the legacy, generic capabilities API. Modern Selenium 4 uses browser-specific `Options` classes (`ChromeOptions` , `FirefoxOptions`) which are type-safe, provide autocomplete, and map to W3C WebDriver protocol capabilities. `ChromeOptions` extends `MutableCapabilities` which extends `DesiredCapabilities` — they are compatible, but use `Options` directly.

35.4 TestNG Rapid Fire

Q: What is the difference between `@BeforeMethod` , `@BeforeClass` , and `@BeforeSuite` ?

A: `@BeforeMethod` runs before each `@Test` method — use for per-test setup (fresh browser, test data). `@BeforeClass` runs once before any test in the class — use for expensive class-level setup (DB connection). `@BeforeSuite` runs once before the entire suite — use for global infrastructure (Grid startup, report directory). Execution order: Suite → Test → Groups → Class → Method.

Q: How does TestNG's `dependsOnMethods` work and what is the risk of using it?

A: `@Test(dependsOnMethods = {"loginTest"})` makes the annotated test skip/fail if the dependency fails. It enforces execution order and creates a dependency graph. The risk: it couples tests together, reduces parallelism, and creates fragile chains where one failure cascades. Prefer test independence — use `@BeforeMethod` for shared setup instead of dependency chains.

Q: What is `ITestListener` and name the key callback methods?

A: `ITestListener` is a TestNG listener interface for responding to test lifecycle events. Key methods: `onTestStart()` , `onTestSuccess()` , `onTestFailure()` , `onTestSkipped()` , `onTestFailedButWithinSuccessPercentage()` , `onStart(ITestContext)` , `onFinish(ITestContext)` . Register via XML `<listeners>` , `@Listeners` annotation, or `ServiceLoader` .

Q: What is the difference between `@DataProvider` and `@Parameters` in TestNG?

A: `@Parameters` injects values from `testng.xml` — static, single values per parameter name. `@DataProvider`

returns an `Object[][]` — each row is a separate test invocation with its own set of arguments, enabling true data-driven testing. `@DataProvider` supports `parallel=true` for concurrent data-driven execution. `@Parameters` cannot drive multiple test invocations.

Q: How do you run TestNG tests in parallel and what are the parallelism modes?

A: Set `parallel` attribute in `testng.xml`: `parallel="methods"` runs each test method in a separate thread. `parallel="classes"` runs each class in a separate thread. `parallel="tests"` runs each `<test>` tag in a separate thread. `parallel="instances"` runs each instance in a separate thread. Combine with `thread-count="N"` to set pool size. Ensure thread-local `WebDriver` management (via `ThreadLocal<WebDriver>`).

Q: What is `IRetryAnalyzer` and how do you implement it without masking real failures?

A: `IRetryAnalyzer` lets you rerun failed tests up to N times. Implement `retry(ITestResult result)` returning `true` to retry or `false` to stop. Risk: it masks genuinely flaky product bugs and inflates pass rates. Best practice: (1) Log every retry with reason. (2) Only retry on known transient exceptions (timeouts, network errors), not assertion errors. (3) Track retry metrics separately in your test report.

Q: What is a TestNG Group and how do you use it for selective test execution?

A: Groups tag tests with labels: `@Test(groups = {"smoke", "regression"})`. In `testng.xml`, include/exclude groups: `<groups><run><include name="smoke"/></run></groups>`. This enables smoke-only runs in pre-deployment pipelines and full regression only in nightly builds. Groups can be inherited across `@BeforeGroup` / `@AfterGroup` hooks. Use consistently — ungrouped tests are hard to selectively execute.

Q: How does TestNG handle test ordering within a class by default?

A: TestNG does NOT guarantee method execution order within a class by default — it may run alphabetically or in declaration order depending on the JVM. Use `@Test(priority = N)` to enforce order (lower priority runs first). Use `preserve-order="true"` in `testng.xml` to maintain XML declaration order. For strict ordering, use `dependsOnMethods` as an ordered chain.

Q: What is `ISuite` and `ITestContext` and when do you access them in listeners?

A: `ISuite` represents the entire test suite — access suite-level attributes, results, and the XML suite configuration. `ITestContext` represents a `<test>` tag in `testng.xml` — access test-level included/excluded groups, parameters, passed/failed/skipped results, and the start/end times. Both are used in `ISuiteListener` and `ITestListener` respectively to build rich custom reporters.

Q: What does `alwaysRun = true` do on a configuration method?

A: `@AfterMethod(alwaysRun = true)` ensures the method runs even if the test method failed or was skipped — critical for teardown (closing browser, releasing connections). Without it, a failed `@BeforeMethod` causes `@AfterMethod` to be skipped, leaking resources. Always set `alwaysRun = true` on `@After*` cleanup methods.

Q: How do you pass parameters from `testng.xml` to a `@DataProvider`?

A: Inject `ITestContext` as a parameter in the `@DataProvider` method — it provides access to the test context's parameters from `testng.xml`:

```
@DataProvider
public Object[][] testData(ITestContext context) {
    String env = context.getCurrentXmlTest().getParameter("env");
    return loadDataForEnv(env);
}
```

Q: What is `SoftAssert` and when should you NOT use it?

A: `SoftAssert` collects all assertion failures and reports them together at `assertAll()`, rather than stopping at the first failure. Do NOT use it when subsequent assertions depend on earlier ones being true (e.g., asserting on an element that may not exist if the prior check failed). Avoid using a single `SoftAssert` across multiple test methods — it must be instantiated per test to avoid cross-contamination.

Q: How do you configure TestNG via code (programmatic) instead of XML?

A: Use the `TestNG` class directly:

```
TestNG testNG = new TestNG();
XmlSuite suite = new XmlSuite();
suite.setName("ProgrammaticSuite");
suite.setParallel(XmlSuite.ParallelMode.METHODS);
suite.setThreadCount(5);
XmlTest test = new XmlTest(suite);
test.setXmlClasses(List.of(new XmlClass(MyTest.class)));
testNG.setXmlSuites(List.of(suite));
testNG.run();
```

Useful for dynamic CI pipeline configurations where XML is generated at runtime.

Q: What is the execution order when both `@BeforeClass` and `@BeforeMethod` are inherited from a parent class?

A: Parent `@BeforeClass` → Child `@BeforeClass` → (for each test) Parent `@BeforeMethod` → Child `@BeforeMethod` → `@Test` → Child `@AfterMethod` → Parent `@AfterMethod` → Child `@AfterClass` → Parent `@AfterClass`. TestNG respects class hierarchy in configuration method ordering.

Q: What is the difference between `@Factory` and `@DataProvider`?

A: `@DataProvider` runs the same test method multiple times with different data. `@Factory` creates multiple instances of a test class, each instantiated with different constructor arguments — each instance runs all its test methods. Use `@Factory` when the entire test class state must vary per scenario (e.g., different base URLs, different user roles initializing the entire page object graph).

35.5 REST Assured Rapid Fire

Q: What is the BDD-style syntax in REST Assured and what does each clause represent?

A: `given()` sets up the request specification (headers, auth, body, params). `when()` specifies the HTTP method and endpoint. `then()` contains response assertions. The full chain: `given().header("X-API-Key", key).body(payload).when().post("/users").then().statusCode(201).body("id", notNullValue())`. This maps directly to Gherkin Given/When/Then for readability.

Q: How do you extract a value from a JSON response in REST Assured?

A: Use `extract().path()` with JsonPath expression:

```
String userId = given().when().get("/users/1")
    .then().statusCode(200)
    .extract().path("data.id");
```

Or extract the full response: `Response response = given().when().get("/users/1").then().extract().response(); String id = response.jsonPath().getString("data.id");`

Q: What is `RequestSpecification` and `ResponseSpecification`? Why use them?

A: `RequestSpecification` (built via `RequestSpecBuilder`) captures reusable request configurations — base URI, headers, auth, content type. `ResponseSpecification` (built via `ResponseSpecBuilder`) captures reusable response assertions — status code, content type, response time. Define them once in a base class and reuse across all tests, avoiding repetition and centralizing API contract assertions.

```
RequestSpecification spec = new RequestSpecBuilder()
    .setBaseUrl("https://api.example.com")
    .addHeader("Authorization", "Bearer " + token)
    .setContentType(ContentType.JSON)
    .build();
```

Q: How do you test OAuth2 bearer token authentication in REST Assured?

A: `given().auth().oauth2(token).when().get("/protected-resource")` — REST Assured adds the `Authorization: Bearer <token>` header automatically. For form-based OAuth flows, use `given().auth().form(username, password, formConfig)`. For the token acquisition step itself, make a separate POST to the token endpoint and extract the `access_token` before the actual test.

Q: What is the difference between `body()` with Hamcrest matchers vs. `body()` with JsonPath?

A: With Hamcrest: `then().body("name", equalTo("John"))` — evaluates the JsonPath expression and applies the matcher in a single assertion. With `extract().body().asString()` + JsonPath manually — gives you the raw string to parse yourself. Hamcrest integration is more concise; manual parsing is needed for complex conditional assertions or when you need to reuse the extracted value.

Q: How do you validate response time in REST Assured?

A: `then().time(lessThan(2000L), TimeUnit.MILLISECONDS)` — asserts the response was received within 2 seconds. Or extract: `long ms = response.getTime()`. This is the round-trip time from `send()` to `receive()` measured by REST Assured's HTTP client, not server processing time alone. Combine with reporting to build SLA violation alerts.

Q: How do you handle multipart file uploads in REST Assured?

A: Use `multiPart()`:

```
given()
    .multiPart("file", new File("/path/to/file.csv"), "text/csv")
    .multiPart("description", "Test file upload")
    .contentType("multipart/form-data")
    .when().post("/upload")
    .then().statusCode(200);
```

REST Assured sets the correct `Content-Type` boundary automatically.

Q: What is `JsonSchemaValidator` in REST Assured and how does it work?

A: It validates the response JSON against a JSON Schema (draft v4/v7).

`then().body(matchesJsonSchemaInClasspath("schemas/user-schema.json"))` — REST Assured loads the schema and validates the response body against it using the `json-schema-validator` library. Catches structural regressions (missing required fields, wrong types) that field-by-field assertions miss.

Q: How do you enable request/response logging in REST Assured for debugging?

A: Use `.log().all()` on request or response:

`given().log().all().when().get("/users").then().log().ifError().log().ifError()` only logs on non-2xx responses — keeps CI logs clean. `log().ifValidationFails()` logs only when an assertion fails. Global logging: `RestAssured.enableLoggingOfRequestAndResponseIfValidationFails()`.

Q: How do you deserialize a JSON response directly into a Java object (POJO) with REST Assured?

A: `User user = given().when().get("/users/1").then().statusCode(200).extract().as(User.class)` — REST Assured uses Jackson or Gson (whichever is on the classpath) to deserialize. Ensure your POJO has matching field names or use Jackson annotations (`@JsonProperty`) for mapping mismatches. Set `RestAssured.config = RestAssuredConfig.config().objectMapperConfig(new ObjectMapperConfig(ObjectMapperType.JACKSON_2))` to be explicit.

Q: What is the difference between query params, path params, and form params in REST Assured?

A: `queryParams("status", "active")` appends `?status=active` to the URL. `pathParam("userId", 123)` replaces `{userId}` in `get("/users/{userId}")` → `/users/123`. `formParam("username", "john")` sends `application/x-www-form-urlencoded` body key-value pairs. Using the wrong one changes the request structure entirely — `pathParam` errors cause 404, `queryParams/formParam` confusion causes 400/422.

Q: How do you chain REST Assured calls where the second request depends on the first response?

A: Extract the required value from the first response and use it in the second:

```
String token = given().body(loginPayload).when().post("/auth/login")
    .then().statusCode(200).extract().path("access_token");

given().auth().oauth2(token).when().get("/profile")
    .then().statusCode(200).body("email", equalTo("test@example.com"));
```

Q: How do you mock an external API in REST Assured integration tests?

A: Use WireMock as a local stub server: start `WireMockServer`, configure stubs with

`stubFor(get(urlEqualTo("/external")).willReturn(aResponse().withStatus(200).withBody(json)))` , then point REST Assured's `baseUrl` to `http://localhost:{wireMockPort}` . WireMock supports request matching, response templating, stateful scenarios, and fault injection (delays, connection drops).

Q: What is `relaxedHTTPSValidation()` and when should you NOT use it in production tests?

A: `given().relaxedHTTPSValidation()` disables SSL/TLS certificate verification — accepts any certificate, including self-signed and expired ones. Use ONLY in local/dev environments with self-signed certs. Never in staging/production tests — it defeats TLS validation entirely and would pass against a man-in-the-middle. For staging, install the test cert into a custom `TrustStore` instead.

Q: How do you parameterize REST Assured tests with environment-specific base URIs?

A: Externalize via system properties or environment variables: `RestAssured.baseUrl = System.getProperty("base.uri", "https://api-dev.example.com")` . Pass via Maven: `mvn test -Dbase.uri=https://api-staging.example.com` . Or use a config library (Owner, Typesafe Config) that maps profiles to property files loaded at test startup, setting `RestAssured.baseUrl` in a `@BeforeSuite` hook.

35.6 Playwright Rapid Fire

Q: What is the fundamental architectural difference between Playwright and Selenium?

A: Playwright communicates with browsers via the Chrome DevTools Protocol (CDP) for Chromium, and analogous low-level protocols for Firefox/WebKit — all compiled into the Playwright engine as patched browser builds. Selenium uses the W3C WebDriver protocol (HTTP-based), sending commands to a separate driver binary. Playwright's direct protocol connection enables faster execution, auto-waiting, network interception, and browser context isolation without a driver intermediary.

Q: What is a `BrowserContext` in Playwright and how is it different from a `Page` ?

A: `BrowserContext` is an isolated browser session — separate cookies, localStorage, cache, and permissions. It is the Playwright equivalent of an incognito window. Multiple `Page` objects can exist within one context (as tabs). Multiple contexts can share one `Browser` instance. Use separate contexts per test for isolation; use multiple pages within a context to simulate multi-tab flows.

Q: What is Playwright's auto-waiting mechanism and how does it differ from explicit waits?

A: Every Playwright action (`click()` , `fill()` , `check()`) automatically waits for the element to be: attached to DOM, visible, stable (no animation), not disabled, and not obscured. This is baked into the action itself — no separate `waitForSelector()` needed in most cases. Explicit `waitForSelector()` or `waitForLoadState()` is only needed for non-action waits (waiting for navigation, network idle, dynamic content before assertion).

Q: What is `page.waitForLoadState()` and what are the three states?

A: `'load'` — waits for the `load` event (all resources including images). `'domcontentloaded'` — waits for the `DOMContentLoaded` event (HTML parsed, scripts deferred). `'networkidle'` — waits until no network requests for 500ms. `'networkidle'` is the most common for SPAs; `'domcontentloaded'` is fastest but may miss dynamic content. Use `'load'` for server-rendered pages with heavy assets.

Q: How does Playwright handle network interception and what can you do with it?

A: `page.route(urlPattern, handler)` intercepts matching requests. From the handler you can: `route.fulfill({body, status})` to mock the response, `route.abort()` to simulate failures, `route.continue()` to pass through with optional modification. Use cases: mocking API endpoints, blocking analytics scripts, simulating network errors, and injecting test data without a real backend.

```
await page.route('**/api/users', route =>
  route.fulfill({ status: 200, body: JSON.stringify([
    {id: 1, name: 'Test'}]) })
);
```

Q: What is `page.evaluate()` and when do you use it?

A: `page.evaluate(fn)` executes a JavaScript function in the browser context and returns the serialized result back to Node.js/Java. Use it to: read DOM properties not exposed via Playwright APIs, set `localStorage` / `sessionStorage` for test state setup, trigger custom events, or read computed styles. `page.evaluateHandle()` returns a `JSHandle` for non-serializable objects that you interact with further.

Q: What is the difference between `locator.click()` and `page.click(selector)` ?

A: `page.click(selector)` is the legacy, selector-based API — it resolves the selector at call time. `locator.click()` uses the `Locator` API — the locator encapsulates the selector and re-resolves it on each action, making it more resilient to DOM changes. `Locator` also supports chaining, filtering, and composing complex selectors. Prefer `Locator` over page methods in all modern Playwright code.

Q: How does Playwright's `Locator` handle multiple matching elements?

A: By default, actions on a `Locator` matching multiple elements throw if more than one element matches a strict action. Use `.first()`, `.last()`, `.nth(index)` to target specific elements, or `.filter({hasText: 'Login'})` to narrow down. `locator.all()` returns all matching `Locator` instances for iteration. `expect(locator).toHaveCount(N)` asserts the exact count.

Q: What is `storageState` in Playwright and how do you use it to skip login in tests?

A: `storageState` captures browser state — cookies, `localStorage`, `sessionStorage` — to a JSON file. After logging in once in a setup fixture, save: `await context.storageState({ path: 'auth.json' })`. In subsequent tests, load it: `const context = await browser.newContext({ storageState: 'auth.json' })`. The browser starts already authenticated, eliminating login UI traversal for every test.

Q: How does Playwright's `expect()` API differ from Chai/Jest assertions for UI testing?

A: Playwright's `expect(locator)` has built-in retry logic — it polls the assertion condition (up to `timeout`, default 5s) until it passes or times out. Standard Chai/Jest `expect` runs once synchronously and fails immediately. This makes Playwright assertions inherently robust against async UI updates without requiring manual waits before asserting. Example: `await expect(page.locator('.success-message')).toBeVisible()` retries automatically.

Q: What is Playwright's `codegen` and what are its limitations for production use?

A: `npx playwright codegen https://example.com` launches a browser with an inspector that records user interactions and generates Playwright test code in real time. Limitations: (1) Generated selectors are often fragile (positional or role-based without context). (2) No assertion generation — you must add `expect()` calls manually. (3)

No data-driven logic. (4) Code must be refactored into proper Page Objects before production use. Treat it as a starting scaffold only.

Q: How do you run Playwright tests in parallel and what is the parallelism model?

A: In `playwright.config.ts`, set `workers: N` (default: CPU count / 2). Each worker is a separate Node.js process with its own browser instance. Tests within a single file run serially by default; set `fullyParallel: true` to run all tests across all files in parallel. Use `test.describe.configure({ mode: 'parallel' })` at describe-block level for finer control. Worker count scales with available CPUs and memory.

Q: What is `page.waitForResponse()` and how do you use it to synchronize with AJAX?

A: `waitForResponse(urlOrPredicate)` returns a Promise that resolves when a matching network response is received. Use it to synchronize test actions with background API calls:

```
const [response] = await Promise.all([
  page.waitForResponse(r => r.url().includes('/api/save') && r.status() === 200),
  page.click('#save-button')
]);
```

This is far more reliable than `waitForTimeout()` or `networkidle` for specific API synchronization.

Q: What is the difference between Playwright's `test.beforeEach()` and `test.beforeAll()` ?

A: `test.beforeEach()` runs before every individual test — use for fresh `Page` and `BrowserContext` creation (isolation). `test.beforeAll()` runs once per worker for all tests in a file — use for expensive shared setup (database seeding, authenticated context creation). Note: `test.beforeAll()` shares state across tests in the file, so mutations must be carefully managed to avoid test interdependencies.

Q: How do you handle browser dialogs (alert, confirm, prompt) in Playwright?

A: Register a `page.on('dialog', handler)` listener before triggering the dialog:

```
page.on('dialog', async dialog => {
  expect(dialog.message()).toBe('Are you sure?');
  await dialog.accept(); // or dialog.dismiss()
});
await page.click('#delete-button');
```

Unlike Selenium's `switchTo().alert()`, Playwright's event-driven model handles dialogs without switching context.

Q: What is Playwright's `APIRequestContext` and when do you use it instead of REST Assured?

A: `APIRequestContext` (accessible via `request` fixture) sends HTTP requests using Playwright's HTTP client, which shares the same cookie jar and authentication state as the browser context. Use it for API setup/teardown within Playwright tests (e.g., creating test data via API before UI steps) without needing a separate REST Assured dependency. For pure API test suites, REST Assured/RestTemplate is still more feature-rich.

35.7 CI/CD Rapid Fire

Q: What is the difference between a CI pipeline and a CD pipeline?

A: CI (Continuous Integration) automatically builds, tests, and validates code on every push — providing fast feedback. CD (Continuous Delivery) extends CI to automatically deploy validated builds to staging/pre-production, ready for manual production release. CD (Continuous Deployment) goes further — automatically deploys to production without human approval. In SDET context: CI runs unit/integration/smoke tests; CD gates include full regression and performance gates.

Q: What is a Jenkins agent and how does it differ from the Jenkins controller?

A: The Jenkins controller (master) manages the web UI, job scheduling, and pipeline orchestration. Agents (formerly slaves) are worker processes that execute build steps on separate machines. The controller offloads execution to agents via JNLP, SSH, or Docker. Agents can be static (always-on), ephemeral (Docker/Kubernetes-spawned per build), or cloud-based (EC2, GCE). Run tests on agents, never on the controller.

Q: What is a `Jenkinsfile` and what are the two pipeline syntaxes?

A: A `Jenkinsfile` is a Groovy-based pipeline-as-code file checked into source control. Declarative syntax uses `pipeline { agent; stages { stage { steps {} } } }` — structured, opinionated, easier to read and validate. Scripted syntax uses `node { stage('name') { ... } }` — full Groovy, more flexible but harder to lint. Declarative is preferred for most test pipelines; use `script {}` blocks inside Declarative when Groovy logic is needed.

Q: What is the difference between `sh` and `bat` steps in a Jenkins pipeline?

A: `sh` executes a shell command on Unix/Linux/macOS agents. `bat` executes a batch command on Windows agents. For cross-platform pipelines, use `isUnix()` to branch, or use Docker containers with a Linux base image to normalize the execution environment regardless of agent OS.

Q: What is a GitHub Actions workflow and what are the key components?

A: A workflow is a YAML file in `.github/workflows/`. Key components: `on` (trigger events — push, pull_request, schedule), `jobs` (parallel units of work), `steps` (sequential actions within a job), `runs-on` (runner OS), `uses` (reusable actions from marketplace), `env / secrets` (environment variables). Jobs run in parallel by default; use `needs: [job-id]` for sequential dependencies.

Q: How do you cache Maven/Gradle dependencies in GitHub Actions to speed up builds?

A: Use the `actions/cache` action keyed on the dependency lock file:

```
- uses: actions/cache@v3
  with:
    path: ~/.m2/repository
    key: ${{ runner.os }}-maven-`${{ hashFiles('**/pom.xml') }}
    restore-keys: ${{ runner.os }}-maven-
```

The cache is restored if the key matches exactly; fallback uses the prefix. This avoids re-downloading dependencies on every run, saving 2-5 minutes per build typically.

Q: What is a Docker multi-stage build and how does it reduce test image size?

A: Multi-stage builds use multiple `FROM` instructions. The first stage (builder) compiles and runs tests with the full SDK. The second stage (runtime) copies only the artifacts into a minimal base image, discarding build tools. For SDET

use: a test-runner image builds with the full Java/Maven/browser stack but only artifacts (reports, logs) are exported, keeping the image lean for subsequent stages.

Q: What is a GitHub Actions matrix strategy and how do you use it for cross-browser testing?

A: Matrix strategy runs the same job with multiple variable combinations:

```
strategy:
  matrix:
    browser: [chromium, firefox, webkit]
    environment: [staging, qa]
```

This creates 6 parallel jobs (3 browsers × 2 environments). Each job receives `${{ matrix.browser }}` and `${{ matrix.environment }}` as context variables passed to the test command. Combine with `fail-fast: false` so all matrix variants complete even if one fails.

Q: What is a quality gate in SonarQube and how does it integrate with CI?

A: A quality gate is a set of conditions (code coverage ≥ 80%, no new critical bugs, duplications < 3%) that a codebase must pass. The `sonar-maven-plugin` analyzes code and posts results to SonarQube Server. The CI pipeline uses `sonar.qualitygate.wait=true` to poll until the analysis completes and fails the build if the gate fails. This prevents code debt from accumulating by blocking PRs that degrade quality metrics.

Q: What is the difference between blue-green deployment and canary deployment?

A: Blue-green: two identical environments (blue=live, green=new). Switch traffic all-at-once from blue to green after testing passes; rollback by switching back. Zero-downtime but doubles infrastructure cost. Canary: route a small percentage (5-10%) of traffic to the new version; gradually increase percentage as monitoring confirms stability. Slower rollout but limits blast radius; requires feature flagging or weighted routing (nginx, Istio).

Q: How do you implement test parallelism in a Jenkins pipeline without race conditions on shared resources?

A: Use `parallel` blocks in Declarative pipelines for concurrent stage execution. Prevent race conditions: (1) Use unique test data per parallel branch (UUID-suffixed). (2) Use separate DB schemas or Testcontainers instances per branch. (3) Use browser Grid with sufficient node capacity. (4) Avoid shared file writes — write reports to unique paths, then merge. (5) Use Jenkins `lock` step for truly shared resources.

Q: What is OWASP Dependency Check and where does it fit in a CI pipeline?

A: OWASP Dependency Check scans project dependencies against the National Vulnerability Database (NVD) for known CVEs. In CI, run it as a post-build step: `mvn org.owasp:dependency-check-maven:check`. Set `failBuildOnCVSS=7` to fail builds with high-severity vulnerabilities. Schedule full NVD database updates in nightly runs; use cached data in PR builds for speed. Block production releases on critical CVEs.

Q: What is a `Dockerfile` `ENTRYPOINT` vs. `CMD` and how does it affect test runner containers?

A: `ENTRYPOINT` defines the executable that always runs — it is not overridden by `docker run` arguments. `CMD` provides default arguments to `ENTRYPOINT` or a default command if `ENTRYPOINT` is not set. For a test runner: `ENTRYPOINT ["mvn"]` with `CMD ["test"]` allows `docker run my-tests -Dgroups=smoke` to override only the args, always using Maven. This enables flexible test parameterization from CI without rebuilding the image.

Q: What is environment parity and why does it matter for SDET pipelines?

A: Environment parity means dev, CI, staging, and production environments are as identical as possible in configuration, OS, dependencies, and data. Lack of parity causes "works on my machine" failures where tests pass locally but fail in CI due to different JVM versions, timezone differences, locale settings, or path separators. Achieve parity via Docker containers, Infrastructure-as-Code (Terraform), and config management (Helm, Ansible).

Q: How do you manage secrets (API keys, passwords) in a CI/CD pipeline securely?

A: Never hardcode secrets in code or `Jenkinsfile`. Use: (1) Jenkins Credentials Store — inject via `withCredentials([string(credentialsId: 'api-key', variable: 'API_KEY')])`. (2) GitHub Actions Secrets — accessed via `${{ secrets.API_KEY }}`. (3) HashiCorp Vault — dynamic secrets with short TTLs. (4) AWS Secrets Manager / Azure Key Vault for cloud-native. Mask secrets in logs; rotate regularly; audit access. The principle of least privilege applies.

35.8 SQL Rapid Fire

Q: What is the difference between `INNER JOIN`, `LEFT JOIN`, and `FULL OUTER JOIN`?

A: `INNER JOIN` returns only rows with matching values in both tables. `LEFT JOIN` returns all rows from the left table and matched rows from the right; unmatched right rows are NULL. `RIGHT JOIN` is the mirror. `FULL OUTER JOIN` returns all rows from both tables — unmatched rows on either side get NULLs. In test data validation: `LEFT JOIN` is essential for finding orphaned records or missing relationships.

Q: What is the difference between `WHERE` and `HAVING`?

A: `WHERE` filters individual rows before grouping — it cannot reference aggregate functions. `HAVING` filters groups after `GROUP BY` — it can reference aggregates. `SELECT status, COUNT(*) FROM test_runs WHERE env='prod' GROUP BY status HAVING COUNT(*) > 5` — `WHERE` filters before grouping, `HAVING` filters aggregate results.

Q: What is a SQL index and what are the tradeoffs of adding one?

A: An index is a B-tree (or other) data structure that speeds up `SELECT` queries on indexed columns by avoiding full table scans — $O(\log n)$ lookup vs $O(n)$. Tradeoffs: write operations (`INSERT`, `UPDATE`, `DELETE`) are slower because the index must be updated. Indexes consume additional disk space. Too many indexes degrade write-heavy workloads (test result insertion). Index selectively — high-cardinality columns used in `WHERE` clauses and `JOIN` conditions.

Q: What is the difference between `TRUNCATE`, `DELETE`, and `DROP`?

A: `DELETE` removes specific rows (with optional `WHERE`) — logged per row, slow on large tables, can be rolled back. `TRUNCATE` removes all rows without logging individual row deletions — much faster, DDL operation in most DBs, cannot be rolled back in many engines, resets auto-increment. `DROP` removes the entire table structure and data. For test teardown: `TRUNCATE` to reset tables, `DELETE WHERE test_run_id = ?` for specific cleanup.

Q: What is a CTE (Common Table Expression) and when do you use it over a subquery?

A: A CTE (`WITH name AS (SELECT ...)`) defines a named temporary result set usable in the main query. Use CTEs over subqueries when: (1) the same subquery is referenced multiple times (CTE is evaluated once). (2) Readability — CTEs name intermediate results semantically. (3) Recursive queries (hierarchical data — test suites containing sub-suites). CTEs do not always improve performance; the optimizer may inline them.

Q: What is a window function and how does it differ from `GROUP BY` ?

A: Window functions (`ROW_NUMBER()` , `RANK()` , `LAG()` , `SUM()` `OVER(...)`) compute aggregates over a sliding window of rows relative to the current row without collapsing rows. `GROUP BY` collapses multiple rows into one per group. Window functions retain individual row detail while adding aggregate context. Example: `ROW_NUMBER() OVER (PARTITION BY suite_id ORDER BY duration DESC)` ranks tests within each suite by duration.

Q: What is `EXPLAIN` / `EXPLAIN ANALYZE` in PostgreSQL and how do you use it?

A: `EXPLAIN` shows the query execution plan (which operations the planner will use — SeqScan, IndexScan, HashJoin) with estimated costs. `EXPLAIN ANALYZE` actually executes the query and shows real timing and row counts alongside estimates. Use it to diagnose slow queries, identify missing indexes (SeqScan on large tables), and verify that the planner uses indexes correctly. Key metrics: `actual time` , `rows` , `Buffers: hit vs read` .

Q: What is the difference between `UNION` and `UNION ALL` ?

A: `UNION` removes duplicate rows — it sorts and deduplicates the combined result set ($O(n \log n)$ overhead). `UNION ALL` retains all rows including duplicates and is faster (no dedup pass). Use `UNION ALL` when you know results are distinct or when duplicates are acceptable (e.g., combining test results from multiple runs where duplicates are meaningful). Only use `UNION` when deduplication is actually required.

Q: What is database normalization and what are 1NF, 2NF, 3NF?

A: Normalization eliminates data redundancy and update anomalies. **1NF:** Each column contains atomic values, no repeating groups. **2NF:** 1NF + no partial dependencies (non-key columns depend on the whole primary key, not part of it — relevant for composite PKs). **3NF:** 2NF + no transitive dependencies (non-key columns depend only on the PK, not on other non-key columns). In test management DBs, 3NF prevents anomalies when updating test suite names or environment configs.

Q: What is a stored procedure vs. a function in SQL?

A: A stored procedure is a named, precompiled block of SQL+procedural code — can modify data (DML), call other procedures, and does not require a return value. A function must return a value (scalar or table-valued) and is side-effect free in theory (cannot perform DDL or transaction control in many DBs). Functions can be used in `SELECT` statements; procedures cannot. Use procedures for complex ETL in test data management; use functions in queries.

Q: What is an `N+1 query problem` and how do you detect and fix it?

A: N+1 occurs when code executes 1 query for a list and then N additional queries for each item (e.g., load 100 tests, then 100 separate queries for each test's steps). Detected via query logging — you see 101 identical queries. Fix: use a `JOIN` or `IN` clause to fetch related data in one query, or use an ORM-level eager loading (`JOIN FETCH` in JPQL, `.includes()` in ActiveRecord). In test reporting DBs, this is a common cause of slow dashboard load times.

Q: What is the difference between `CHAR` , `VARCHAR` , and `TEXT` data types?

A: `CHAR(n)` is fixed-length — always stores n characters, padded with spaces. Fast for fixed-size codes. `VARCHAR(n)` is variable-length up to n characters — no padding. `TEXT` is unlimited variable-length. For test automation DBs: use `VARCHAR(255)` for test names and status codes, `TEXT` for full error messages and stack traces. `CHAR` is appropriate only for fixed codes (e.g., `CHAR(2)` for country codes).

Q: How do you write a SQL query to find duplicate records in a table?

A: Use `GROUP BY` with `HAVING COUNT(*) > 1` :

```
SELECT test_name, suite_id, COUNT(*) as count
FROM test_cases
GROUP BY test_name, suite_id
HAVING COUNT(*) > 1
ORDER BY count DESC;
```

To get the full row details, join back: `SELECT * FROM test_cases WHERE (test_name, suite_id) IN (SELECT test_name, suite_id FROM test_cases GROUP BY test_name, suite_id HAVING COUNT(*) > 1)` .

Q: What is a transaction isolation level and what anomalies does each level prevent?

A: Four levels: **Read Uncommitted** — allows dirty reads (sees uncommitted changes). **Read Committed** (default in most DBs) — prevents dirty reads. **Repeatable Read** — prevents dirty + non-repeatable reads (same SELECT returns same rows within transaction). **Serializable** — prevents all anomalies including phantom reads. Higher isolation reduces concurrency. For test data setup, use `READ COMMITTED` ; for financial validation queries, use `SERIALIZABLE` .

Q: Write a query to get the second-highest test execution duration.

A:

```
-- Method 1: Subquery
SELECT MAX(duration_ms)
FROM test_executions
WHERE duration_ms < (SELECT MAX(duration_ms) FROM test_executions);

-- Method 2: Window function (handles ties correctly)
SELECT duration_ms
FROM (
    SELECT duration_ms, DENSE_RANK() OVER (ORDER BY duration_ms DESC) AS rnk
    FROM test_executions
) ranked
WHERE rnk = 2
LIMIT 1;
```

`DENSE_RANK()` handles ties correctly; the subquery method may return wrong results when there are ties.

35.9 Automation Framework Rapid Fire

Q: What is the Page Object Model (POM) and what are its key design rules?

A: POM encapsulates page UI elements (locators) and page actions (methods) in dedicated classes. Rules: (1) No assertions inside Page Objects — they return values or page objects, not pass/fail. (2) Page methods return Page Objects to enable fluent chaining. (3) Locators are private fields. (4) Business-readable method names (`loginAs(user)` , not `clickLoginButton()`). (5) One Page Object per distinct page/component.

Q: What is the Screenplay Pattern and how does it improve on POM?

A: Screenplay models tests as Actors performing Tasks using Abilities, making goals (outcomes) explicit rather than procedural steps. `actor.attemptsTo(Login.as(user), Navigate.to(Dashboard.class))`. Compared to POM: more readable (business-language tasks), better separation of concerns (who does what vs. how), and scales better for complex multi-actor scenarios (API + UI + DB interactions). Popular with Serenity BDD.

Q: What is the Factory Method pattern in test framework design?

A: Factory Method provides an interface for creating objects but lets subclasses/configuration decide which class to instantiate. In frameworks: `DriverFactory.getDriver(browserType)` returns a `ChromeDriver`, `FirefoxDriver`, or `RemoteWebDriver` based on configuration without exposing instantiation logic to test classes. This decouples tests from concrete driver implementations and makes adding new browsers trivial.

Q: What is the Builder pattern and how is it used in test framework objects?

A: Builder separates complex object construction from representation, enabling readable construction of objects with many optional parameters. In frameworks:

```
ApiRequest request = new ApiRequest.Builder()
    .endpoint("/users")
    .method(HttpMethod.POST)
    .header("Content-Type", "application/json")
    .body(userPayload)
    .timeout(5000)
    .build();
```

Eliminates telescoping constructors and makes test data object creation self-documenting.

Q: What is the Singleton pattern and what is wrong with using it for WebDriver?

A: Singleton ensures only one instance of a class exists. Using a Singleton `WebDriver` breaks parallel test execution — all parallel threads share one browser, causing race conditions and test interference. Solution: use `ThreadLocal<WebDriver>` — each thread gets its own `WebDriver` instance, stored in thread-local storage, providing isolation without a static singleton.

```
private static final ThreadLocal<WebDriver> driver = new ThreadLocal<>();
public static WebDriver getDriver() { return driver.get(); }
public static void setDriver(WebDriver d) { driver.set(d); }
```

Q: What is the Strategy pattern in the context of test data management?

A: Strategy defines a family of algorithms, encapsulates each, and makes them interchangeable. In test data management: `interface TestDataStrategy { TestData load(String testId); }` with implementations `DatabaseStrategy`, `JsonFileStrategy`, `ApiStrategy`, `FakerStrategy`. The test class depends only on `TestDataStrategy` — switching data sources requires only changing the injected strategy, not test code.

Q: What is a test fixture and how does it differ from a test helper?

A: A test fixture is the full set of preconditions and context required for a test — database state, authentication tokens, browser setup, test data. In Playwright: `test.extend()` fixtures. In JUnit 5: `@ExtendWith`. A test helper is a utility

class providing reusable low-level operations (formatting dates, generating UUIDs, reading JSON). Fixtures manage lifecycle (setup/teardown); helpers are stateless utilities.

Q: How do you design a test framework to support multiple environments (dev, staging, prod)?

A: Use a layered configuration system: (1) Default config in `application.yaml`. (2) Environment-specific overrides in `application-{env}.yaml`. (3) System property / env var overrides at the top layer. Use a config library (Owner, Typesafe Config, Spring Boot profiles) to merge layers. Map environment to base URIs, credentials, feature flags, and timeout values. Activate via `mvn test -Denv=staging` or `ENV=staging` in CI.

Q: What is the difference between data-driven and keyword-driven testing?

A: Data-driven: same test logic runs with multiple data sets — the framework drives test parameterization (e.g., `@DataProvider`, CSV files). Keyword-driven: test logic is abstracted into reusable keywords (actions) — non-technical users define test scenarios using keyword tables (Selenium IDE, Robot Framework). Data-driven scales for business data variations; keyword-driven lowers the technical barrier for QA analysts. Hybrid approaches combine both.

Q: What is `TestContainers` and what problem does it solve in integration testing?

A: Testcontainers launches real Docker containers (MySQL, PostgreSQL, Redis, Kafka, WireMock, etc.) within JUnit/TestNG tests, providing disposable, isolated service instances. It eliminates dependency on shared test environments, makes tests fully self-contained, and ensures environment parity. `@Container` and `@Testcontainers` annotations handle container lifecycle. Works with `@DynamicPropertySource` in Spring Boot to wire dynamic ports.

Q: How do you implement test reporting with Allure and what are the key annotations?

A: Add `allure-testng` dependency + AspectJ agent. Key annotations: `@Epic` (top-level grouping), `@Feature` (functional area), `@Story` (user story), `@Step` (auto-logs method as test step), `@Attachment` (adds screenshots/logs to report), `@Severity(SeverityLevel.CRITICAL)` (test priority), `@Description` (human-readable doc). Run `allure serve target/allure-results` to view. Attach screenshots in `ITestListener.onTestFailure()`.

Q: What is `ExtentReports` and when would you choose it over Allure?

A: ExtentReports generates self-contained HTML reports that need no server to view — just open the HTML file. Allure requires either the Allure CLI server or a CI plugin. Choose ExtentReports for simple, shareable single-file reports for non-technical stakeholders. Choose Allure for rich, interactive reports with history trends, retry analysis, and deep CI integration. ExtentReports is simpler to set up; Allure is more powerful for large teams.

Q: How do you handle test environment teardown failures without masking the actual test failure?

A: In TestNG, use `@AfterMethod(alwaysRun=true)` — but wrap the teardown body in try-catch and log failures without rethrowing. Store the original test failure separately. In JUnit 5, use `@AfterEach` with `TestInfo` and catch teardown exceptions to log them as suppressed. The key is: teardown failures should be flagged as warnings/errors, but the original test failure result must be preserved and reported accurately.

Q: What is the Hexagonal Architecture (Ports and Adapters) pattern and how does it apply to test framework design?

A: Hexagonal architecture separates business logic (core domain) from external systems (ports/adapters). In test

frameworks: the "core" is your test logic and domain model (Page Objects, test steps). Ports are interfaces:

`BrowserPort` , `ApiPort` , `DatabasePort` . Adapters are implementations: `SeleniumAdapter` , `PlaywrightAdapter` , `RestAssuredAdapter` , `JdbcAdapter` . Tests depend only on ports — swapping Selenium for Playwright requires only a new adapter.

Q: How do you measure and track test suite health metrics beyond pass/fail?

A: Track: (1) **Flake rate** — $(\text{tests passing inconsistently} / \text{total tests}) \times 100$. (2) **Execution time trend** — per-suite duration over builds. (3) **Mean Time to Detect (MTTD)** — time from bug introduction to test failure. (4) **Coverage delta** — lines/branches covered per build vs. prior. (5) **Retry rate** — % of tests that pass only after retry. (6) **Test age** — tests not modified in >6 months are stale. Surface via Grafana dashboards fed from test result APIs or DB.

35.10 Senior SDET Scenario Rapid Fire

Q: Your flaky test suite has an 8% flake rate. Walk me through your systematic approach to reduce it below 1%.

A: Phase 1 — **Measure**: Instrument tests with retry count, failure reason, and timestamp. Tag retried tests separately. Build a flake dashboard (by test, by browser, by environment). Phase 2 — **Categorize**: Classify flakes by root cause — async timing issues (40%), environment instability (20%), test data conflicts (20%), locator fragility (10%), infrastructure (10%). Phase 3 — **Fix by category**: (1) Timing: Replace `Thread.sleep()` with explicit waits on specific conditions. (2) Data: Isolate test data per test with UUID-suffixed records or Testcontainers. (3) Locators: Replace XPath with stable `data-testid` attributes. (4) Environment: Run against dedicated QA instances, not shared. Phase 4 — **Quarantine**: Move confirmed flaky tests to a `flaky` group — exclude from blocking pipeline, fix iteratively. Phase 5 — **Prevention**: Add a flake budget policy — any test with >2% flake rate in 30-day window gets quarantined automatically. Target: <1% in 6 weeks.

Q: Your CI pipeline takes 45 minutes. How do you get it under 10 minutes?

A: Step 1 — **Profile**: Use pipeline timing data to find the biggest bottleneck (usually test execution). Step 2 — **Parallelize tests**: Split test suite across matrix jobs (by module, by feature, by test type). Run 5 parallel workers → 5× speedup on tests alone. Step 3 — **Cache aggressively**: Cache Maven/npm dependencies (save 3-5 min), Docker layer caching (save 2-3 min on image builds). Step 4 — **Fail fast**: Run unit tests → smoke tests first; only run full regression if they pass. Step 5 — **Optimize test selection**: Use test impact analysis — only run tests affected by changed code paths. Step 6 — **Upgrade infrastructure**: Larger CI runners (8 CPU vs 2 CPU) halve execution time. Step 7 — **Eliminate redundancy**: Remove duplicate test coverage between unit/integration layers. Realistically: Parallelism + caching alone typically gets 45 min → 8-12 min for most suites.

Q: How do you design a test strategy for a system with no existing tests?

A: Step 1 — **Risk assessment**: Map the system's critical paths (highest business value, highest failure risk). These are tested first. Step 2 — **Layer the pyramid**: Define the target split — 70% unit, 20% integration/API, 10% E2E. Don't start with E2E. Step 3 — **Contract tests**: Establish API contracts (OpenAPI specs, Pact contracts) immediately to catch regressions at the interface level. Step 4 — **Smoke suite first**: Build a 10-15 test smoke suite covering happy paths of the 5 most critical user journeys. Get it into CI within Week 1. Step 5 — **Coverage baseline**: Run code coverage analysis to identify completely untested critical code. Prioritize these. Step 6 — **Defect seeding**: Analyze bug history (if available) to identify historically buggy areas — test those first. Step 7 — **Living document**: Document the test strategy — coverage goals, excluded areas (rationale), ownership, review cadence. Iterate monthly.

Q: A test that was passing for 3 months suddenly starts failing in CI but not locally. How do you debug it?

A: Hypothesis-driven debugging: (1) **Environment diff:** Compare CI environment (JVM version, browser version, OS) against local. Check if any dependency was updated. (2) **Timing:** Was the CI runner upgraded or is it under higher load? Add timing logs — maybe an explicit wait timeout is being hit. (3) **Data state:** Is CI using shared test data that may have been mutated by another test or team? Run with fresh data. (4) **Parallelism:** Did the number of parallel workers change? Classic race condition — a shared resource is now contended. (5) **Feature flag/config change:** Was a new feature toggled on in the CI environment? (6) **Reproduce in CI:** Add `debug` logs to the test, push to a branch, observe CI output. Never accept "works locally" — the CI failure is the truth. (7) **git bisect** the change that first introduced the failure to narrow scope.

Q: You are asked to build a cross-browser, cross-device test automation framework from scratch for a greenfield project. What are your architectural decisions?

A: Tool selection: Playwright (multi-browser built-in, auto-waiting, network interception, API testing, TypeScript-first). **Architecture:** Page Object Model with optional Screenplay Pattern for complex flows. **Structure:** `pages/` , `tests/` , `fixtures/` , `utils/` , `data/` , `config/` . **Config management:** `playwright.config.ts` with project-based browser configs; environment loaded from `.env` files via `dotenv` . **Data strategy:** API-driven setup, Faker.js for dynamic data, `storageState` for auth. **Reporting:** Playwright HTML report + Allure for CI integration + Slack notification on failure. **Parallelism:** Worker-per-test with `fullyParallel: true` , 4-8 workers in CI. **Devices:** Playwright device emulation for mobile viewports; real device testing via BrowserStack for critical paths. **CI:** GitHub Actions matrix across Chromium/Firefox/WebKit + desktop/mobile. **Quality gates:** >85% pass rate required; flake budget <2%; max execution time 15 min for smoke.

Q: Your API test suite has 200 tests but no contract testing. A backend team deploys a breaking change that isn't caught until it hits production. How do you prevent this?

A: Implement **Pact contract testing** (Consumer-Driven Contract Testing). (1) Consumer (frontend/API test suite) defines expectations as Pact contracts — generated automatically during consumer tests. (2) Contracts are published to **Pact Broker**. (3) Provider (backend) runs `pact:verify` against the broker's contracts as part of its own CI pipeline. (4) Backend CI fails if its implementation violates any consumer contract. (5) **Can-I-Deploy** gate: Before any deployment, query the Pact Broker — only deploy if all consumers have verified compatibility with this version. This means the backend team cannot merge a breaking change without their CI catching the contract violation, completely decoupling release timing while guaranteeing compatibility.

Q: You need to test a system that sends emails and SMS notifications. How do you test these external-channel integrations without spamming real users?

A: Email: Use **Mailhog** or **Mailpit** (local SMTP trap) — configure the app to send to the local trap server, then query its API to verify email content, recipient, subject. In CI: run Mailhog as a Docker container. For cloud: use **Mailtrap**. **SMS:** Use **Twilio Test Credentials** (magic numbers that never send real SMS) or mock the SMS gateway behind a feature flag. **In-house:** Create a dedicated `NotificationPort` interface — stub it in unit/integration tests; only integration-test the adapter against the real gateway in a controlled isolated environment. Never use real production phone numbers or emails in automated tests.

Q: How do you test a microservice that depends on 5 external downstream services?

A: Three-layer approach: (1) **Unit tests with mocks:** Mock all 5 downstream clients (Mockito, WireMock) — test service logic in isolation. (2) **Contract tests:** Pact contracts between this service (consumer) and each downstream (provider) — verify interfaces without full integration. (3) **Component tests with stubs:** Run the service with

WireMock stubs for all 5 dependencies — test realistic end-to-end request flows within the service's boundary. (4)

Integration tests (limited): Test against real downstream services only for the most critical paths, in a dedicated integration environment. This layered approach gives fast feedback, realistic validation, and avoids flakiness from real network calls in the main CI pipeline.

Q: Your test database is getting corrupted between test runs, causing intermittent failures. How do you architect data isolation?

A: Gold standard: Use Testcontainers — each test class or test method gets a fresh, disposable container. No shared state possible. **Alternative 1:** Database transactions — wrap each test in a `@Transactional` block that rolls back after the test, restoring pre-test state. **Alternative 2:** Schema-per-test — each test creates a new schema with a UUID name, tears it down after. **Alternative 3:** Use an in-memory DB (H2) for tests that don't need production-specific SQL features. **For shared environments:** Implement a test data manager — each test inserts data tagged with a `test_run_id`, and `@AfterMethod` deletes `WHERE test_run_id = ?`. Never write tests that depend on data created by other tests.

Q: The QA team complains that test failures are hard to diagnose. What improvements do you implement?

A: Multi-layered diagnostics: (1) **Screenshot on failure** — full-page screenshot attached to report in `onTestFailure()`. (2) **Video recording** — Playwright `video: 'retain-on-failure'`, Selenium with Monte Screen Recorder. (3) **Network HAR file** — capture all network requests/responses for the failing test. (4) **Browser console logs** — capture JS errors via `driver.manage().logs().get(LogType.BROWSER)`. (5) **Test step logging** — `@Step` annotations in Allure so the report shows exactly which step failed. (6) **Environment info** — log browser version, OS, JVM, app version, and test data IDs in the report header. (7) **Clear assertion messages** — `assertEquals(actual, expected, "User ID should match the created user's ID")`. Target: a developer should be able to diagnose a failure from the report alone, without re-running the test.

Q: You are leading a team of 4 SDETs. How do you govern test quality and prevent test code from becoming unmaintainable?

A: Structural governance: (1) **Test code reviews** mandatory — same standards as production code. (2) **Test coding standards** document — naming conventions, POM rules, data handling, assertion patterns. (3) **Static analysis** — SonarQube rules for test code (no `Thread.sleep`, no magic strings, no empty catch blocks). (4) **Architecture tests** — ArchUnit rules enforced in CI (`tests must not depend on other tests`, `page objects must not contain assertions`). **Process:** (5) **Test debt backlog** — dedicated sprint capacity (20%) for test maintenance. (6) **Coverage thresholds** — enforced in CI; coverage drops block merges. (7) **Monthly test health review** — flake rates, duplication, execution time trends. **Culture:** (8) **Inner-source model** — developers write and own unit/integration tests; SDETs own E2E framework and strategy. (9) **Blameless test post-mortems** — learn from escapes without finger-pointing.

35.11 Quick Reference Cheat Sheet

Java Core

- `==` compares references; `.equals()` compares values — always use `.equals()` for Strings
- `volatile` ensures visibility but NOT atomicity — use `AtomicInteger` for compound ops
- `synchronized` acquires the monitor lock and establishes happens-before relationships
- Checked exceptions: must declare or catch. Unchecked (`RuntimeException`): optional

- `Optional.orElseGet(supplier)` is lazy; `orElse(value)` always evaluates — prefer `orElseGet`
- `ThreadLocal<WebDriver>` — the correct solution for parallel WebDriver management
- `try-with-resources` requires `AutoCloseable`; suppressed exceptions accessible via `getSuppressed()`
- `StringBuilder` for string building in loops; `String` is immutable (new object per concat)

Collections

- `hashCode()` contract: equal objects MUST have equal hash codes
- `HashMap` default: capacity 10, load factor 0.75, resizes at 75% fill to 1.5× capacity
- Java 8+ `HashMap`: buckets >8 entries + table ≥64 slots → converts linked list to Red-Black tree
- Fail-fast iterators: throw `ConcurrentModificationException` on concurrent structural modification
- Fail-safe iterators (`CopyOnWriteArrayList`): iterate over snapshot, O(n) writes
- `Comparable` = natural ordering (intrusive); `Comparator` = external ordering (non-intrusive)
- Never use `Stack<T>` — use `Deque<T>` `stack = new ArrayDeque<>()`
- `EnumSet` uses bit vectors — fastest possible Set for enum values
- `List.of()` is truly immutable (no nulls, no mutation); `Collections.unmodifiableList()` is a view

Selenium

- `findElements()` returns empty list if no match; `findElement()` throws `NoSuchElementException`
- `driver.quit()` kills session; `driver.close()` closes current window only
- `StaleElementReferenceException` → DOM changed; re-locate the element
- Implicit wait + explicit wait = unpredictable cumulative timeouts — use only explicit waits
- JS clicks bypass browser event pipeline — use only as last resort
- `WebDriverWait` = `FluentWait<WebDriver>` with defaults; `FluentWait` is fully configurable
- `sendKeys("/path/to/file")` on `<input type="file">` for file upload — no Robot/AutoIT needed
- `Actions.perform()` executes the built chain — don't forget it
- Switch context with `driver.switchTo().frame()` before interacting with iframe elements

TestNG

- Configuration execution order: Suite → Test → Groups → Class → Method (same order for `@Before/@After`)
- `@AfterMethod(alwaysRun=true)` — always add this to ensure teardown runs even after test failure
- `@DataProvider(parallel=true)` for concurrent data-driven execution
- `@Factory` creates multiple class instances; `@DataProvider` runs same method with multiple data sets
- `SoftAssert` must be instantiated per test; call `assertAll()` at the end
- `IRetryAnalyzer` — only retry on transient exceptions, NOT assertion failures
- `parallel="methods"` → each method in own thread; `parallel="tests"` → each `<test>` tag in own thread

REST Assured

- `given()` = request setup; `when()` = HTTP action; `then()` = assertions
- Use `RequestSpecBuilder` for reusable base specs across all API tests
- `relaxedHTTPSValidation()` disables SSL cert check — NEVER use in production/staging
- `orElseGet` equivalent in REST Assured: `extract().path()` for typed extraction
- `matchesJsonSchemaInClasspath()` for structural validation — catches missing fields and type errors
- `log().ifValidationFails()` — keeps CI logs clean while still capturing failures
- `page.route()` in Playwright = `WireMock.stubFor()` in REST Assured tests

Playwright

- `BrowserContext` = isolated session (cookies, storage); multiple `Page`s per context
- Auto-wait baked into every action — waits for visible, stable, enabled, not-obscured
- `waitForLoadState('networkidle')` — best for SPAs; `'domcontentloaded'` fastest
- `storageState` — save auth state once, reuse across tests → skip login in every test
- `expect(locator).toBeVisible()` retries automatically up to timeout — no manual wait before assertion
- `page.waitForResponse()` with `Promise.all()` — correct way to sync with AJAX calls
- `Locator` API > `page.click(selector)` — locator re-resolves on each action, more resilient
- `page.route()` for network mocking; `route.fulfill()` to stub, `route.abort()` to simulate errors

CI/CD

- Declarative Jenkinsfile: `pipeline { agent; stages { stage { steps } } }` — preferred
- `actions/cache` keyed on `hashFiles('**/pom.xml')` — cache invalidates when deps change
- Matrix strategy: runs job N×M times for N×M variable combinations — perfect for cross-browser
- Docker `ENTRYPOINT` + `CMD`: `ENTRYPOINT` always runs; `CMD` provides default overridable args
- Secrets: Jenkins Credentials Store / GitHub Secrets — never hardcode in pipeline files
- Quality gate in SonarQube: `sonar.qualitygate.wait=true` fails build if gate fails
- Blue-green: all-at-once switch, easy rollback. Canary: gradual traffic shift, limits blast radius

SQL

- `WHERE` filters rows (before GROUP BY); `HAVING` filters groups (after GROUP BY, can use aggregates)
- `TRUNCATE` = fast bulk delete (DDL, resets auto-increment); `DELETE` = logged per-row, rollbackable
- `INNER JOIN` = intersection; `LEFT JOIN` = all left + matched right (NULLs for unmatched)
- `UNION` deduplicates (slower); `UNION ALL` retains duplicates (faster) — use `UNION ALL` by default
- `EXPLAIN ANALYZE` in PostgreSQL: see actual timing, row counts, buffer hits vs reads
- Window functions (`ROW_NUMBER`, `RANK`, `LAG`) retain row detail while computing aggregates
- N+1 problem: 1 query for list + N queries for each item → fix with `JOIN` or eager loading
- `DENSE_RANK()` handles ties correctly in "find Nth highest" problems

Framework Design

- POM rule: No assertions inside Page Objects — tests assert, page objects act
- `ThreadLocal<WebDriver>` — mandatory for parallel Selenium/Playwright test execution
- Builder pattern — use for complex test data objects with many optional parameters
- Factory Method — `DriverFactory.getDriver(browser)` decouples tests from concrete driver classes
- Strategy pattern — `TestDataStrategy` interface with DB/JSON/API/Faker implementations
- Testcontainers — gold standard for isolated integration test dependencies (DB, Kafka, Redis)
- Flake rate target: <1%; quarantine tests >2% flake rate; never let retries mask real failures
- Test pyramid: 70% unit / 20% integration+API / 10% E2E — never invert the pyramid

Key Scenario Answers

- **Reduce flake rate:** Measure → Categorize root causes → Fix by category → Quarantine → Prevent
- **Speed up CI (45 → 10 min):** Parallelize tests + cache deps + fail-fast ordering + test impact analysis
- **Greenfield test strategy:** Risk-map → pyramid plan → smoke suite in Week 1 → coverage baseline
- **Contract testing:** Pact (Consumer-Driven) → publish to Pact Broker → provider verifies → Can-I-Deploy gate

- **External service isolation:** Testcontainers for DB; WireMock for HTTP; mock ports for SMS/email
 - **Testability governance:** SonarQube + ArchUnit rules + mandatory test code review + test debt budget
 - **Failure diagnostics:** Screenshot + video + HAR + console logs + step logging + clear assertion messages
-

End of Section 35 — Rapid-Fire Interview Questions